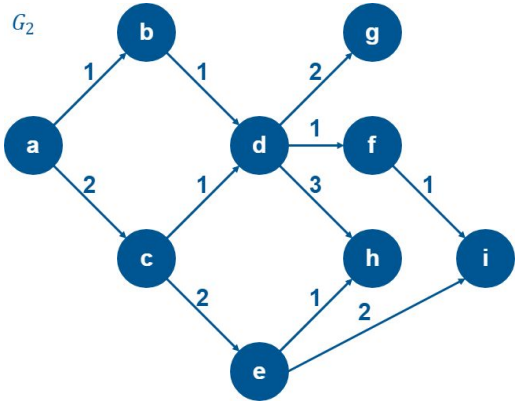
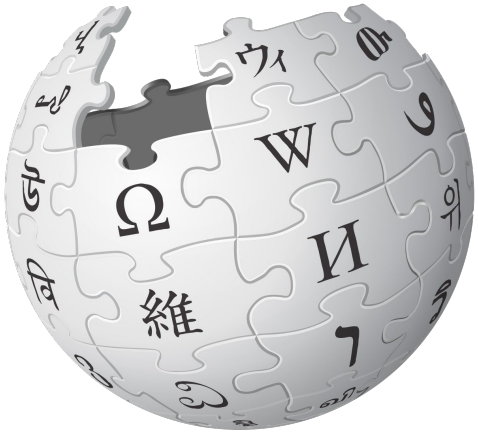
Graph Processing: WikiRace



3P93 Project Step 4
Group 20



GROUP MEMBERS

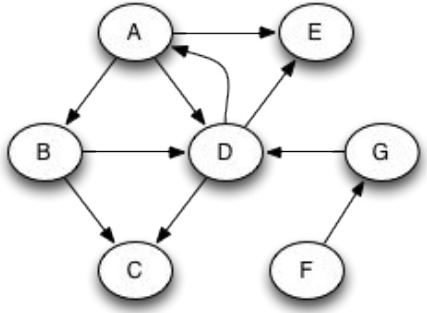
Name	Student #
Geoffrey Jensen	7148710
Nicholas Parise	7242530
Stephen Stefanidis	7140030

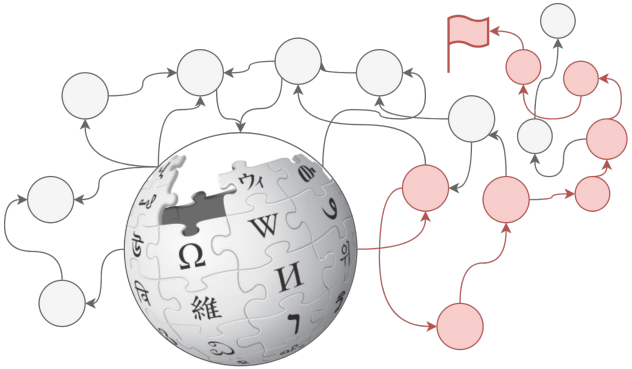
What are we trying to do?

Inspired by Wikipedia Race, our program attempts to find the fastest path between two articles.

Our project is to explore parallel computing strategies to process the **Wikipedia** database. Specifically traversing, and calculating PageRank values.

To achieve this several patterns were used: producer–consumer, data parallelism, task parallelism, OpenMP/OpenMPI directives, ect.





WikiRace

Every article is connected

For instance, these two random articles:

the Moon Parallel computing

They seem completely unconnected, but in Wikipedia there is **always** a path.

The moon

Aristotle

$$\begin{array}{l} p \\ \underline{p \rightarrow q} \\ \therefore q \end{array}$$

Logic

Parallel computing

Problem Size

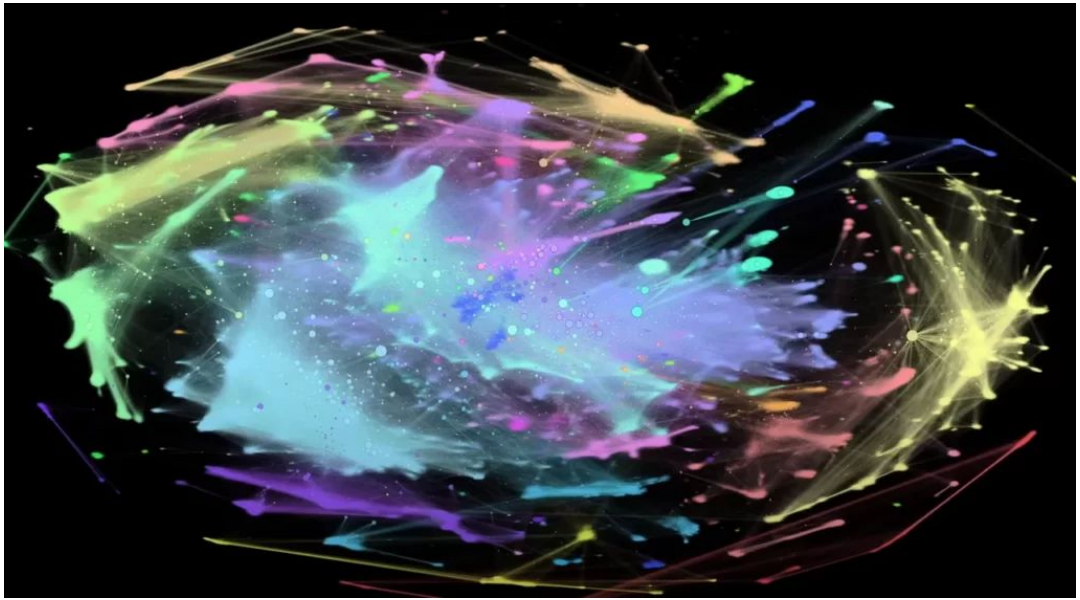
Why do we need data parallelism?

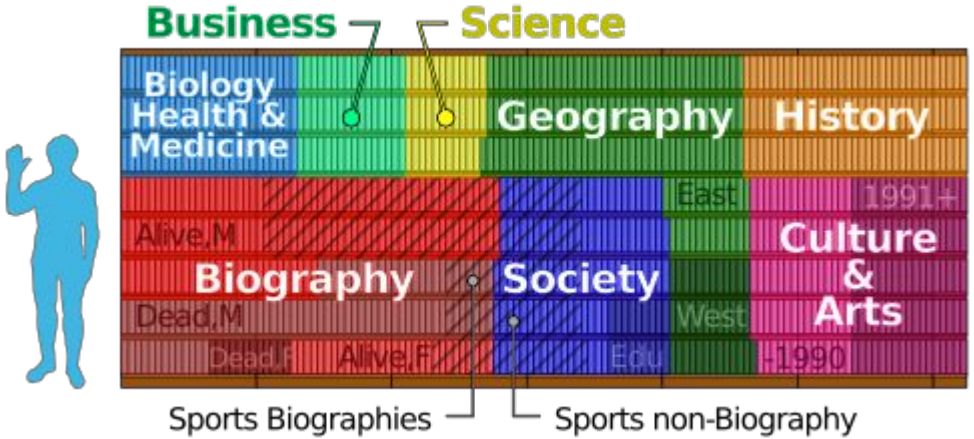
Wikipedia dataset:

- 18+ million pages
- 700+ million links

All Stored in SQLite for fast paging and direct querying.

- 9GB sized uncompressed database





Tools and Libraries Used for Coding;

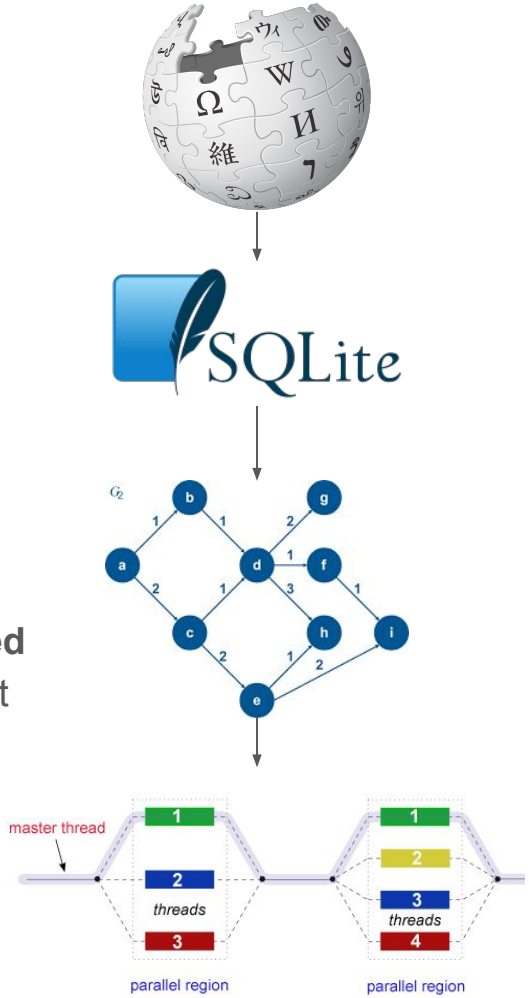
- Written in **C++**
- **SQLITE** (custom compiled with options) to store page data and links
- **OPENMP** & **OPENMPI** for parallel loops and reduction
- **Pthreads** for database producer and consumer implementation
- **CMAKE** for easy compiling on different platforms

DataBase Import / Graph Creation

Creating the database was a big problem in step 1 and 2. Significant time was spent transforming the raw data into a usable form for our application.

Steps to parallelize:

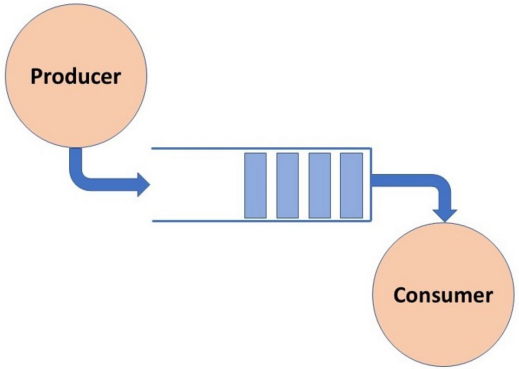
- SQLITE is limited to **one** read thread
  - the import is limited to the **speed that the rows can be processed**
- Each row can be processed **independently**, and therefore is the perfect place to start threading the application.



DataBase Read Algorithm

The parallel database import:

- One pthread for the producer that adds to queue
- OPENMP for consumers that pop from queue and add to local map
- Finally take all local maps and put into finished graph




```
//producer:
while(DB has data){
  row = row_from_database()
  Lock(queue_mutex)
  queue.push(row)
  Unlock(queue_mutex)
}
//consumer:
#pragma omp parallel
{
while(not done){
  if(queue not empty){
    Lock(queue_mutex)
    row = queue.top()
    queue.pop()
    unlock(queue_mutex)
    }
  parsed_row = parse(row)
  local_map.push(parsed_row)
  }
}
// put it all together
for(l_map : local_maps){
  finished_map += l_map
}
```

DataBase Read Performance

- Runs performed on 8 core system, 32GB RAM

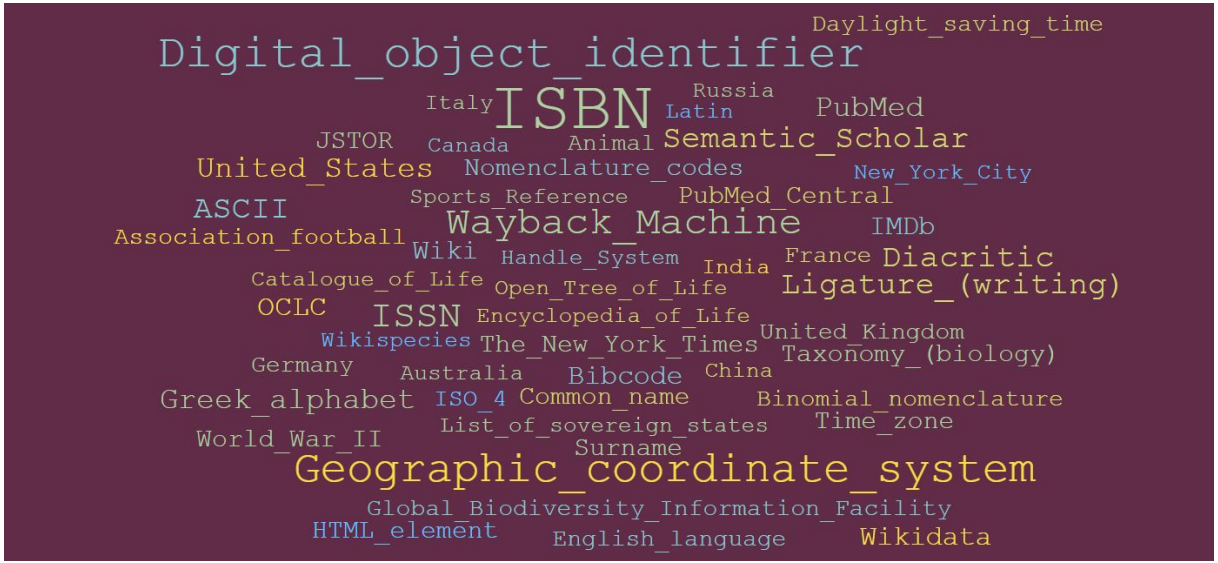
Execution Time (in seconds)		
Number of processes	Time (in seconds)	Improvement
Sequential performance	$\sim$ 20 seconds	
Parallel: 1 worker	$\sim$ 15 seconds	25% improvement
Parallel: 2 worker	$\sim$ 11 seconds	45% improvement
Parallel: 4 worker	$\sim$ 9 seconds	55% improvement
Parallel: 8 worker	$\sim$ 12 seconds	40% improvement

Table 1. Execution Times of Database Read

- Bottleneck at 8 worker threads, most likely due to hardware limitation.

PageRank

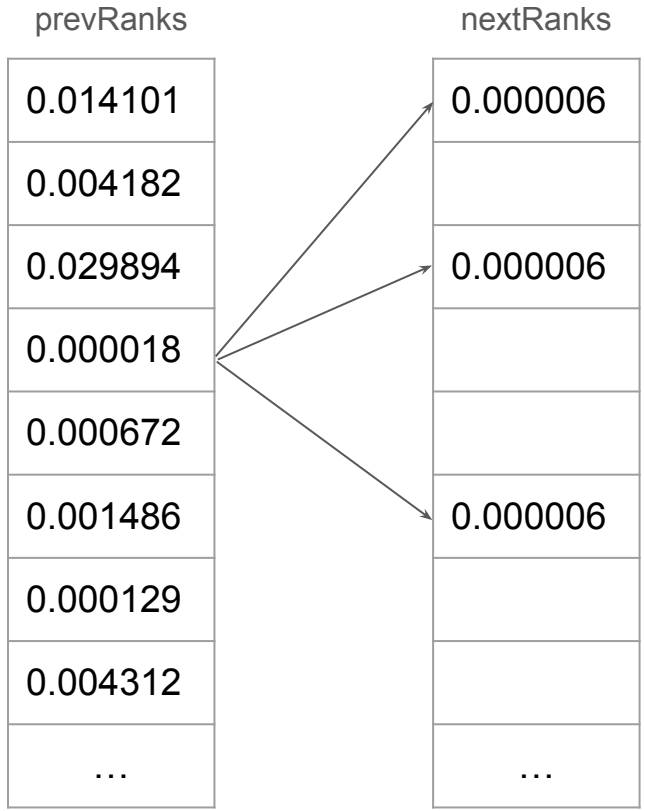
- Developed By Google
- Rank pages by **importance**
  - Create a probability distribution
- Simulates a web-random surfer



Top 52 Wikipedia Articles by PageRank

PageRank Algorithm

- 2 Arrays of length |V|
  - prevRanks
  - nextRanks
- prevRanks initialized as 1 / |V|
- Repeat iteratively until convergence:
  - Disperse page probability of each node among it’s neighbours
- To check convergence:
  - Sum of abs(prevRanks[i] - nextRanks[i]) for all indices

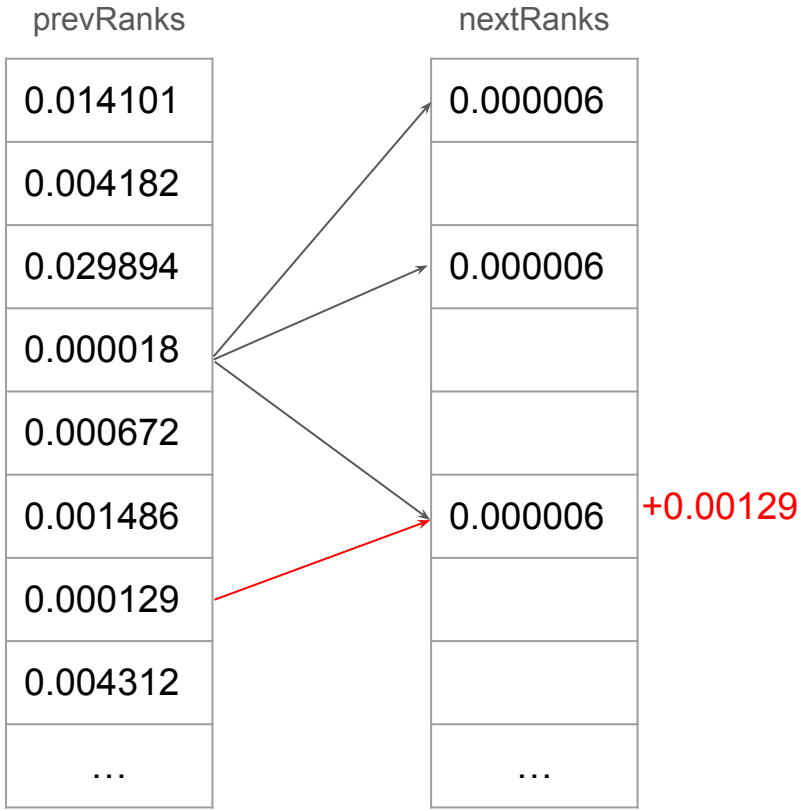


PageRank OpenMP

- #pragma omp parallel for
  - Directly parallelize?

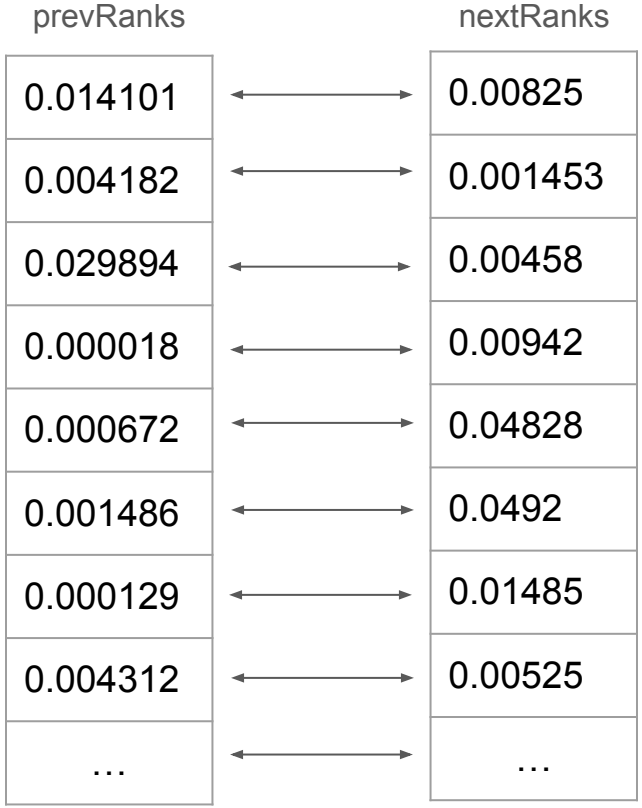
- Synchronization required!
- 18,000,000 Pages
  - 18,000,000 locks

- Change Adjacency List
  - Outgoing -> Incoming connections
- Use dynamic schedule
  - Inlinks in range [0, 1 597 701]



PageRank OpenMP

- Convergence check
- #pragma omp parallel for
  - reduction(+:diff)



PageRank OpenMP Performance

- Runs performed on 6 core system, 16GB RAM

Speedup

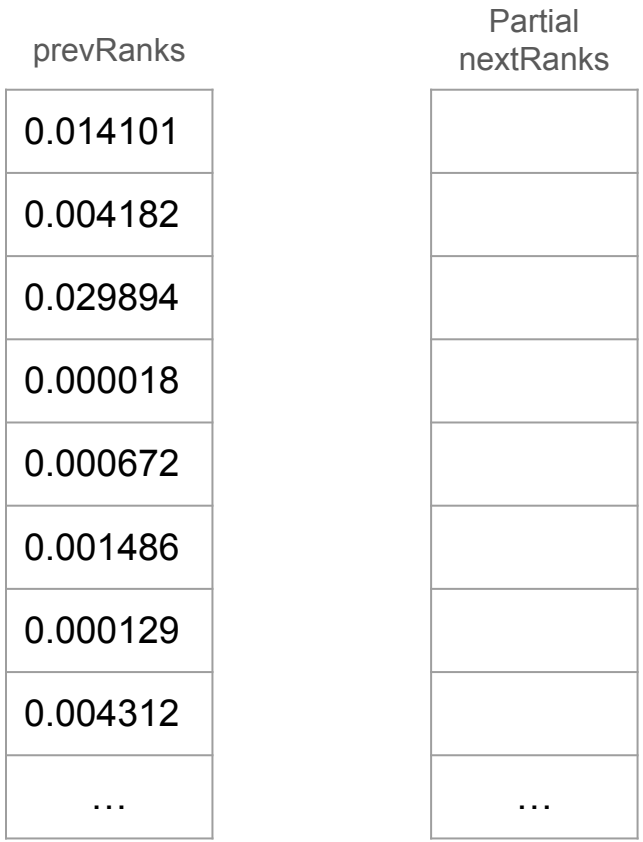
	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	1.82	2.92	3.77	4.31
Half	1.00	1.83	2.85	3.81	4.40
Original	1.00	1.84	2.86	3.85	4.44

Efficiency

	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	0.91	0.73	0.47	0.27
Half	1.00	0.91	0.71	0.48	0.27
Original	1.00	0.92	0.71	0.48	0.28

PageRank MPI

- Every node has a partition of the graph
- Assume every node has the complete prevRanks array
- Calculate local nextRanks based on local partition
- Use MPI_Allreduce to synchronize nextRanks array
- Convergence check
  - Sum of abs(prevRanks[i] - nextRanks[i]) for partitioned nodes
  - MPI_Allreduce for complete value



PageRank MPI Performance Metrics

Execution Times

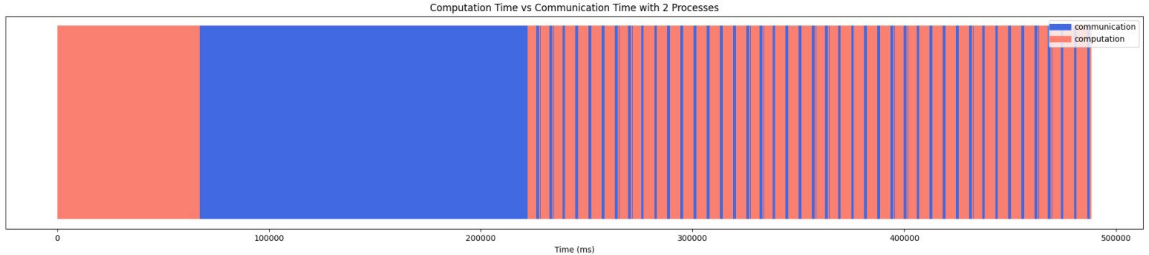
	Number of Processes			
PB	1	2	4	8
Quarter	180	193	197	203
Half	279	300	302	312
Original	523	489	459	469

Speedup

	Number of Processes			
PB	1	2	4	8
Quarter	1.00	0.93	0.91	0.88
Half	1.00	0.93	0.92	0.89
Original	1.00	1.07	1.14	1.12

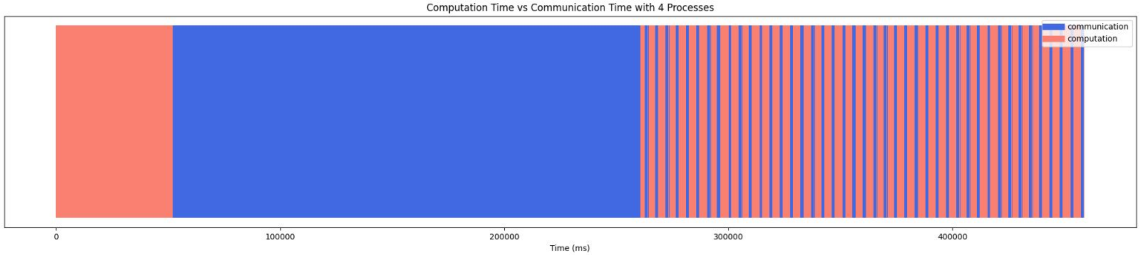
PageRank MPI Computation Time vs Communication Time

2 Processes



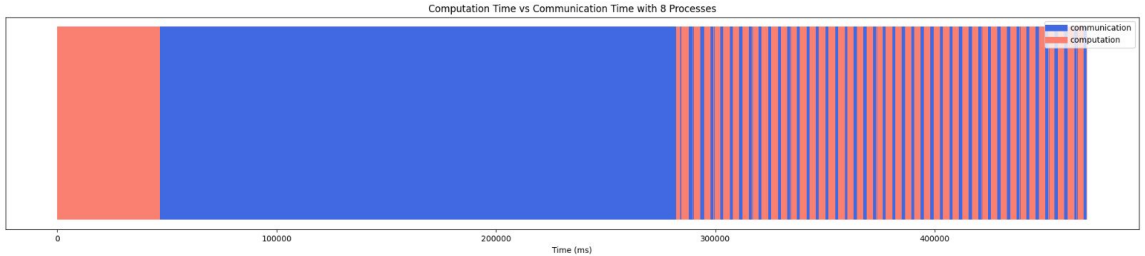
Computation: 56.35%
Communication: 43.65%

4 Processes



Computation: 41.87%
Communication: 58.13%

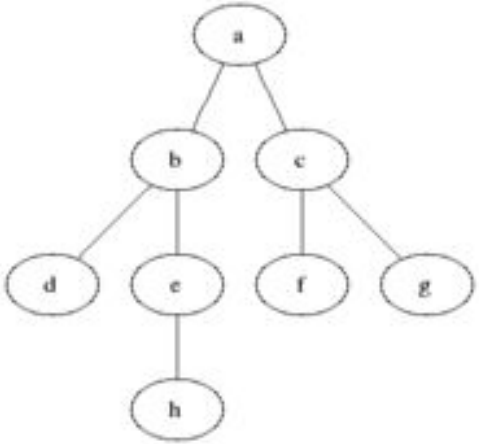
8 Processes



Computation: 36.56%
Communication: 63.44%

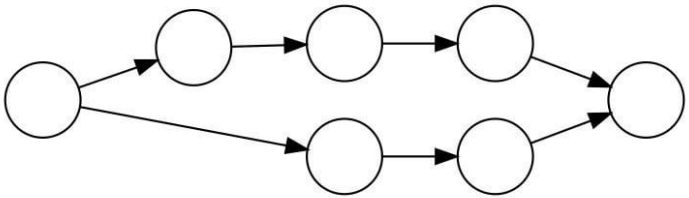
BFS Algorithm

- Classic Search algorithm
- Explores the graph ‘level by level’
- This property leads to BFS always either
  - finding an optimal path
  - exploring all nodes and deciding there is no path



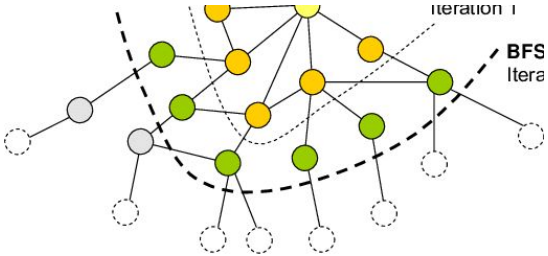
Bidirectional Algorithm

- A variant of BFS that has two ‘symmetrical’ searches
  - One searcher starts from the start
  - The other starts from the end
- Goal is to find which middle node the two searchers meet at
- In the sequential algorithm, you alternate between finishing a level with the forward search and finishing a level with the backward search



BFS Parallelism in Practice

- BFS
  - Expanding the nodes for the current depth level (frontier) can be done in parallel
  - Parallelism is limited by updating the visited/parent data and pushing nodes to the next frontier
- Bidirectional BFS
  - Two searches can in theory explore simultaneously (forward and backward BFS)
  - If we knew exactly which node was the optimal middle node, they could run completely independent of each other
    - However, they need to access the other directions visited list to find the middle node
  - Therefore, this does not offer much improvement due to synchronization overhead



BFS OpenMP - Design and Implementation Decisions

- Used OpenMP parallel for loops when expanding the current frontier, because nodes are independent

```
std::vector<NodeState> next_frontier; // stores nodes on next level
#pragma omp parallel for schedule(dynamic) num_threads(NUM_THREADS)// uses multiple t
for (int i = 0; i < levelSize; i++) {
    if (found) continue; // makes loop end early when a path has already been found.
    NodeState node;
    node = frontier[i];
```

- Used OpenMP sections for creating two task-parallel threads in bidirectional BFS

```
#pragma omp parallel sections shared(done, meeting)
{
    // front bfs expansion
    #pragma omp section
    {
```

OpenMP Problems

- Parent map and next frontier vector are both critical regions that need an OpenMP Critical Section.
  - Creates a large synchronization bottleneck
- Node expansion is not embarrassingly parallel due to depending on the visited node list
- Bidirectional directions cannot both fully run in parallel due to the same critical sections

BFS OpenMPI

- For distributed memory computing, the concept is less about exploring nodes in parallel and more about partitioning the graph across the ranks
- Each rank has its own local frontier and local set of vertices
- Rank 0 creates the graph, partitions and sends each piece to the other ranks
  - (and also keeps a piece)
- In order to use a hybrid approach, you could use OpenMP to expand each local frontier
- While expanding, the ranks fill buffers of the vertices that are not local
- After a level is complete, an All-to-all communication occurs that lets each rank know which vertex of theirs was explored by the others

OpenMPI Problems

- More difficult to implement - explicit message calls are required
- Running out of memory
  - Careful partitioning of the graph is required because the Wikipedia graph is so massive
  - You cannot send the entire graph to each rank!
- BFS does not translate so well to distributed memory computing because a lot of synchronization is required, which adds latency
- Partitioning and distributing the graph can take a long time alone

OpenMPI Functions

- MPI_Bcast
  - Used for sending single variables such as the start and end node
- MPI_Send & MPI_Recv
  - Used for sending and receiving the graph partitions
- MPI_Barrier
  - Used for level synchronization
- MPI_Alltoall
  - Used for comparing the size of next frontier across the ranks
- MPI_Alltoallv
  - Used for comparing next frontiers across the ranks

BFS OpenMP Performance

- Each configuration used 5 sets of random source and destination articles, and recorded the average
- Ideal amount of threads for my computer ends up being 4-8
- BFS is more memory-bound than compute-bound meaning you hit diminishing returns of increasing threads (plus overhead from synchronization and communication)

Execution Times (in seconds)

	Number of Threads				
PB	1	2	4	8	16
Quarter	12.4	6.8	6	3.8	4.8
Half	21.4	10	6	7.2	6
Original	26.8	22.6	9.4	10	11.2

Table 15. Execution Times of BFS OpenMP

Speedup

	Number of Threads				
PB	1	2	4	8	16
Quarter	1.00	1.82	2.07	3.26	2.58
Half	1.00	2.14	3.57	2.97	3.57
Original	1.00	1.19	2.85	2.68	2.39

Table 16. Speedup of BFS OpenMP

Efficiency

	Number of Threads				
PB	1	2	4	8	16
Quarter	1.0	0.91	0.52	0.41	0.16
Half	1.0	1.07	0.89	0.37	0.22
Original	1.0	0.60	0.71	0.34	0.15

Table 17. Efficiency of BFS OpenMP

Group Member Responsibilities and Role

Geoffrey Jensen:

- PageRank implementation (sequential and parallel)
- Performance metrics

Nicholas Parise:

- Graph Database preprocessing (parse scripts and SQL scripts)
- Parallel database loading

Stephen Stefanidis

- BFS implementation (sequential and parallel)
- Bidirectional BFS (sequential and parallel)

All members contributed to debugging, performance measurements, report and the slideshow

The End